Miniproject 1: Image generation with a diffusion model

Jing Wen, Nikolay Chikin

University of Geneva

Jing.Wen@etu.unige.ch
Nikolay.Chikin@etu.unige.ch

December 18, 2023

Overview

1. Context and problem
2. Background on diffusion model
3. Theory and implementation
4. Result and Discuss

Context and problem



- Goal: To build a diffusion model from scratch to generate images.
- We take inspiration from *ElucidatingtheDesignSpaceofDiffusion – BasedGenerativeModels*, *apaper* by Tero Karras, Miika Aittala, Timo Aila, Samuli Laine, presented at NeurIPS 2022, available here.

How to generate images?

We have the following two processes:

- Noising process: start from clean images, add noise until it is indistinguishable from pure noise.
  todo: picture:from clean to noise
- Denoising process: start from pure noise, iteratively denoiser until it looks like an actual image from the data distribution.
  todo: picture: from clean to noise

Noising process

We can gradually degrade an image with a sequence of increasing noise levels $\theta_0 = 0 < \theta_1 < \ldots < \theta_T = \theta_{max}$, in a way that the last step is indistinguishable from pure noise.

Algorithm Noising procedure

Sample an image $y \sim p_{data}$ (from the dataset)
Sample a noise level $\sigma \sim p_{noise}$
Sample gaussian noise $\epsilon \sim N(0, I)$
Obtain degraded sample x by adding noise to the image, $x = y + \sigma\epsilon$

Denoising objective

We are looking for a denoising function D that returns the clean image given the noised image $D(x, \sigma) = y$. Note that D is also taking the noise level σ.

$$\mathcal{L}(\theta) = \mathbb{E}_{y,\sigma,\epsilon}\left[\|D_\theta(y + \sigma\epsilon, \sigma) - y\|^2 \right] \tag{1}$$

where $y \sim p_{data}, \sigma \sim p_{noise}, \epsilon \sim \mathcal{N}(0, I)$ are sampled as described in the previous section.

Parameterization and actual loss for the network

Karras et al. propose the following parameterization of D_θ,
$D_\theta(x, \sigma) = c_{skip}(\sigma)\, x + c_{out}(\sigma)\, F_\theta(c_{in}(\sigma)\, x, c_{noise}(\sigma))$ Where F_θ is the neural network we want to train.

Injecting this parameterization in the denoising loss, we obtain the actual training loss for the network F_θ,

$$\mathcal{L}(\theta) = \mathbb{E}_{y,\epsilon,\sigma}\Big[\| \underbrace{F_\theta\big(c_{\text{in}}(\sigma)\, (y + \sigma\epsilon), c_{noise}(\sigma)\big)}_{\text{Neural network prediction}} - \underbrace{\frac{1}{c_{\text{out}}(\sigma)}\big(y - c_{skip}(\sigma)\, (y + \sigma\epsilon}_{\text{Neural network training target}}$$

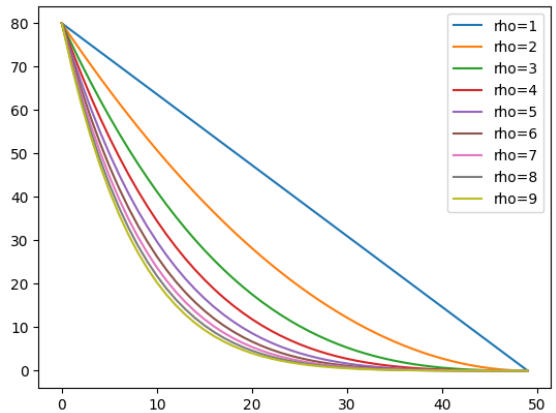
Sampling

When sampling, we are actually solving the following differential equation, called Probability Flow ODE

$$dx = -\dot{\sigma}(t)\sigma(t)\nabla_x \log p(x, \sigma(t)dt$$

where $\nabla_x \log p(x, \sigma)$ is also known as the "score" function.

```
def build_sigma_schedule(steps, rho=7, sigma_min=2e-3, sigma_max=80):
    min_inv_rho = sigma_min ** (1 / rho)
    max_inv_rho = sigma_max ** (1 / rho)
    sigmas = (max_inv_rho + torch.linspace(0, 1, steps) * (min_inv_rho - max_inv_rho)
    return sigmas
```



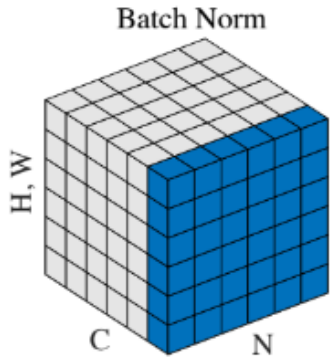
A simple sampling algorithm (Euler's method)

use Euler's method, implemented below.

The datasets

- FashionMNIST:
  methods: Padding: Add extra edge pixels around the image so that can increase image size
  Data normalization: Image normalization, mean = 0.5,std=0.5
- celebA
  methods:CenterCrop: Crop at the center of the image, Keep key information
  Resize: Resize the image, square, side length 178
  Data normalization: Image normalization, mean = 0.5,std=0.5

CondBatchNorm2d



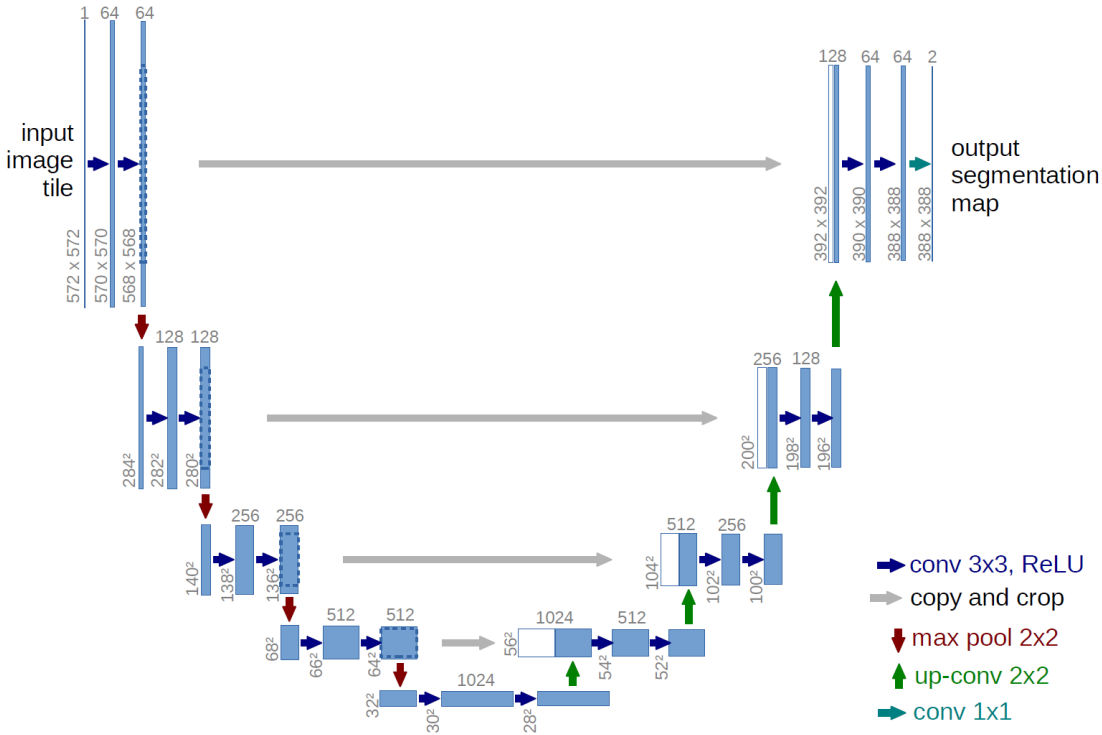
- Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. Input:(N, C, H, W); Output:(N, C, H, W)
- Math:

$$y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} * \gamma + \beta$$

- CondBatchNorm2d: CondBatchNorm2d is a conditional batch normalization layer that can adapt its normalization parameters based on an additional conditional input.
- affine=False: A standard batch normalization layer (nn.BatchNorm2d)

CondResNet

CondUNet



- UNet: Encoder-pooling-decoder architecture to learn feature representations of input images.
- CondUNet: Conditional channels are introduced to generate images based on given conditions.
  Conditional information is integrated at each stage of the encoder and decoder.
- if $cond_channels > 0$, Embed noise information into the model

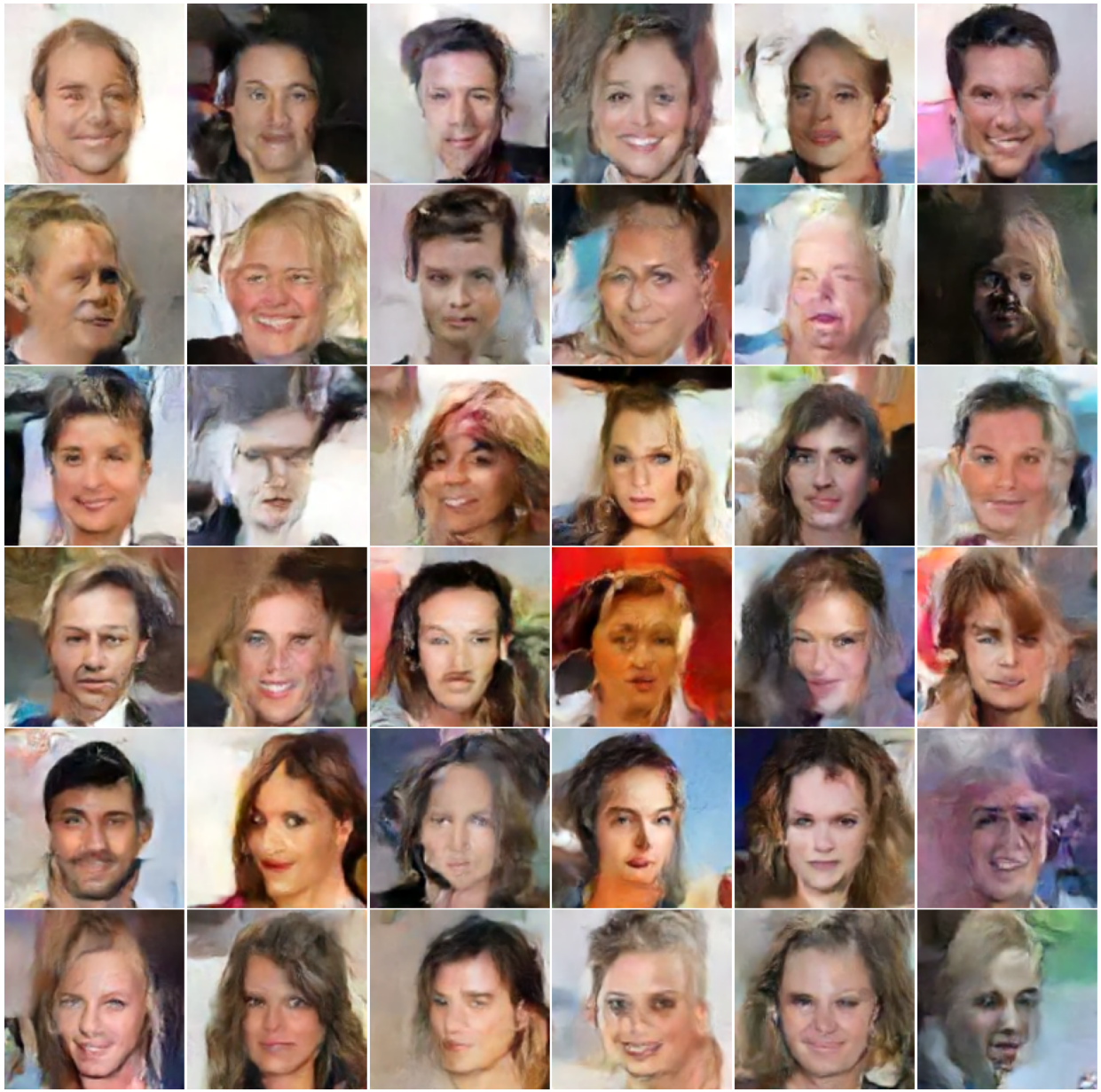
The loss Trends

todo: loss pictures with different models follow the epochs discuss:

The generate images



The generate images



The FID score

FashionMNIST:

- ResNet FID score: 32.97
- CondResNet FID score: 30.75
- ZeroResNet FID score: 29.86
- UNet FID score: 30.37
- CondUNet FID score: 30.77
- ZeroUNet FID score: 31.02

CelebA:

- CondUNet FID score: 35.85

The End

Appendix: ...